Armário Inteligente

Disciplina de IoT – Proposta do projeto (2º semestre/2026)

1. Identificação

Integrantes: Jhonattan Curtarelli; Matheus Augusto; Matheus Mantovani
Título: Armário Inteligente

2. A aplicação

Problema e onde acontece: em academias, os armários costumam usar cadeado ou chave. O aluno esquece a chave, perde o cadeado, e a recepção não tem como saber quais armários estão ocupados ou se alguém deixou a porta aberta. A ideia é resolver isso num armário de academia, sem precisar de chave.

Quem usaria: o aluno (guarda e retira as coisas pelo celular) e a recepção/administrador da academia (acompanha todos os armários e pode liberar um armário se o aluno perder o acesso).

O que o sistema faz:

- O aluno abre o site, lê o QR do armário pela câmera do celular (ou digita o código, ex.: A07) e o celular fica pareado com o armário.
- O armário destrava e o aluno tem 2 min para guardar as coisas e fechar a porta (na demo, 30 s).
- Ao fechar, o ESP32 trava a porta e usa o sensor de distância para ver se tem algo dentro: com objeto fica OCUPADO, vazio volta para LIVRE.
- Ocupado, só abre com o celular pareado (token) ou com a senha do administrador.
- Se o aluno retirar tudo e fechar, o armário volta para LIVRE e o token é apagado.
- Um LED mostra o estado (verde livre, amarelo pareado, vermelho ocupado) e o painel do admin mostra todos os armários.
- As regras de travar/destravar rodam no próprio ESP32, então continuam funcionando se a internet cair.

3. Componentes

Item	Modelo	Função
Placa	ESP32 (DevKit)	Processa as regras e tem Wi-Fi
Sensor	Chave deslizante (simula o reed switch)	Saber se a porta está aberta ou fechada
Sensor	HC-SR04	Distância até o fundo: tem objeto ou não
Atuador	Servo motor SG90	Trinco que trava/destrava a porta
Atuador	3 LEDs (verde, amarelo, vermelho)	Mostrar o estado no armário
Outros	3 resistores de 220 Ω	Proteger os LEDs

Todos esses componentes existem no Wokwi. Como o Wokwi não tem reed switch, vamos usar uma chave deslizante no mesmo pino para simular a porta abrindo e fechando.

4. Comunicação e dados

Como: Wi-Fi do ESP32 + **MQTT** (publish/subscribe) num broker na nuvem (pensamos no HiveMQ Cloud, com usuário e senha). Escolhemos MQTT porque é leve e várias telas (site e painel) podem receber o mesmo estado.

Tópicos (prefixo academia/armario/A07/), mensagens em JSON:

- **estado** (ESP32 envia): {"id":"A07","estado":"ocupado","porta":"fechada","travado":true,"distancia_cm":11.8}
- **evento** (ESP32 envia): ex. token inválido, porta aberta por muito tempo
- **cmd** (site/admin envia): {"acao":"parear","token":"..."} e {"acao":"abrir","token":"..."}; o admin abre com a senha dele

Pareamento: sem servidor próprio. O site gera um token no navegador e guarda no celular; o ESP32 guarda o mesmo token na memória. Ele só obedece o comando se o token bater. Depois de 3 tokens errados ele ignora comandos por 1 min.

Onde o usuário vê o resultado: (1) site do aluno, hospedado no GitHub Pages, com o estado do armário; (2) painel do admin com a tabela de todos os armários e histórico de eventos; (3) LEDs no próprio armário.

5. Montagem

Vamos fazer o projeto **somente no Wokwi**, sem componentes físicos. Na apresentação vamos mostrar a simulação rodando (código compilando, sensores respondendo e dados circulando pelo MQTT), e também

teremos um vídeo da demo como reserva. Link do projeto no Wokwi: https://wokwi.com/projects/479606793513063425.

Limitações que já conhecemos: o HC-SR04 não detecta objetos muito pequenos (moeda, chave) nem tecido muito macio; e o armário não fecha a porta sozinho, então ele só trava depois que o aluno fecha, e a recepção vê no painel quando a porta ficou aberta. Preferimos manter simples e fazer funcionar.

6. O que falta fazer

O esqueleto do código já roda no Wokwi com as regras locais e publica o estado por MQTT, mas ainda usa um broker público de teste. Falta:

- **Conectar a um Pub/Sub real:** criar a conta no broker na nuvem (HiveMQ Cloud) e configurar usuário, senha e TLS, e trocar o broker de teste por ele. Também vamos separar as credenciais num arquivo fora do repositório.
- **Dashboard para os administradores:** uma página web que assina os tópicos de estado de todos os armários, mostra a tabela (livre, pareado ou ocupado, porta, distância) e o histórico de eventos, e tem um botão para abrir ou liberar um armário (publica o comando no MQTT).
- **Site do aluno:** a página que lê o QR (ou recebe o código), faz o pareamento e abre o armário pelo celular.
- **Testes:** testar os fluxos completos no Wokwi (pareamento, ocupado, abrir, token errado, queda do Wi-Fi) e gravar o vídeo da demo.